

- in practice, using SVD of the data matrix for numerical stability
 - keep top- k principal components
 - discard components with small eigenvalues that often correspond to noise
 - Explained variance ratio
 - $\frac{\sum_{i=1}^k \lambda_i}{\sum_j \lambda_j}$
 - minimize mean squared reconstruction error among all rank- k linear projections
 - **Limitations**
 - sensitive to feature scaling
 - cannot capture nonlinear manifolds
-

Deep Learning

Convolutional Neural Networks (CNN)

- convolution kernels with weight sharing
- local receptive field → local information
- translation equivariance
- parameter-efficient compared to fully connected layers
 - same kernel → applied across spatial locations
- downsampling
 - reducing spatial resolution
 - pooling
 - avoid heavy pooling by using strided convolutions
 - stride & padding

- $(H_p - kH) // \text{stride} + 1$

- batch normalization → optimization ↑ (regularizer)
- # of parameters: $C_{\text{out}} \cdot C_{\text{in}} \cdot kH \cdot kW$

Depth-wise Convolution

- using a kernel for each input channel
- # of parameters: $C_{\text{in}} \cdot kH \cdot kW$

Point-wise Convolution

- # of parameters: $kH \cdot kW$

Depthwise Separable Convolution

- $k_H k_W C_{\text{in}} + C_{\text{in}} C_{\text{out}}$

Average Adaptive Pooling

- `nn.AdaptiveAvgPool2d((1,1))`
 - commonly used to remove spatial dimension before a FC layer
 - variable input size
 - fixed-size representation
-

RNN

$$h_t = f(W_x x_t + W_h h_{t-1} + b)$$

- sequence modeling architecture for sequential data
 - explicitly models temporal dependencies

- Transformer
 - attention-based global context
 - fully parallelizable
 - better long-range dependency modeling
-

ResNet

- CNN architecture built from residual blocks
 - enables training of very deep networks
- key idea
 - learn a residual function instead of a full mapping
 - output: $y = F(x) + x$
 - where $F(x)$ is a stack of conv + norm + activation layers
- why it helps (intuition)
 - if the optimal mapping is close to identity, it is easier to learn $F(x) \approx 0$
 - gradients can flow through the skip path directly → vanishing gradient ↓
- typical block types
 - basic block: two 3×3 conv layers
 - bottleneck block: $1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$ (compute efficient)
- common details
 - if shapes differ, use a projection shortcut
 - $y = F(x) + W_s x$ (e.g., 1×1 conv with stride)

Residual Connection

$$y = x + F(x)$$